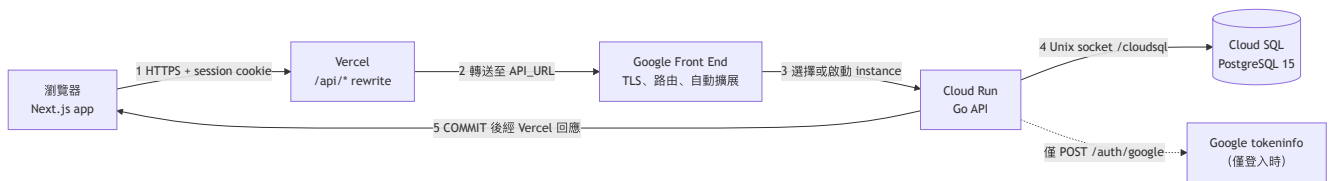


# 1. 應用程式架構

Go-Split 用來分攤一場活動的團體開銷。主辦人建立活動、以邀請碼邀請成員、記錄開銷，最後一次結算。主辦人輸入時，瀏覽器會即時預覽每筆分攤；以伺服器算出的結果為準。

本文件分為兩個視角：**執行期視角**呈現一個請求如何穿過運行中的系統，**建置與部署視角**呈現各元件從何而來、如何上線。

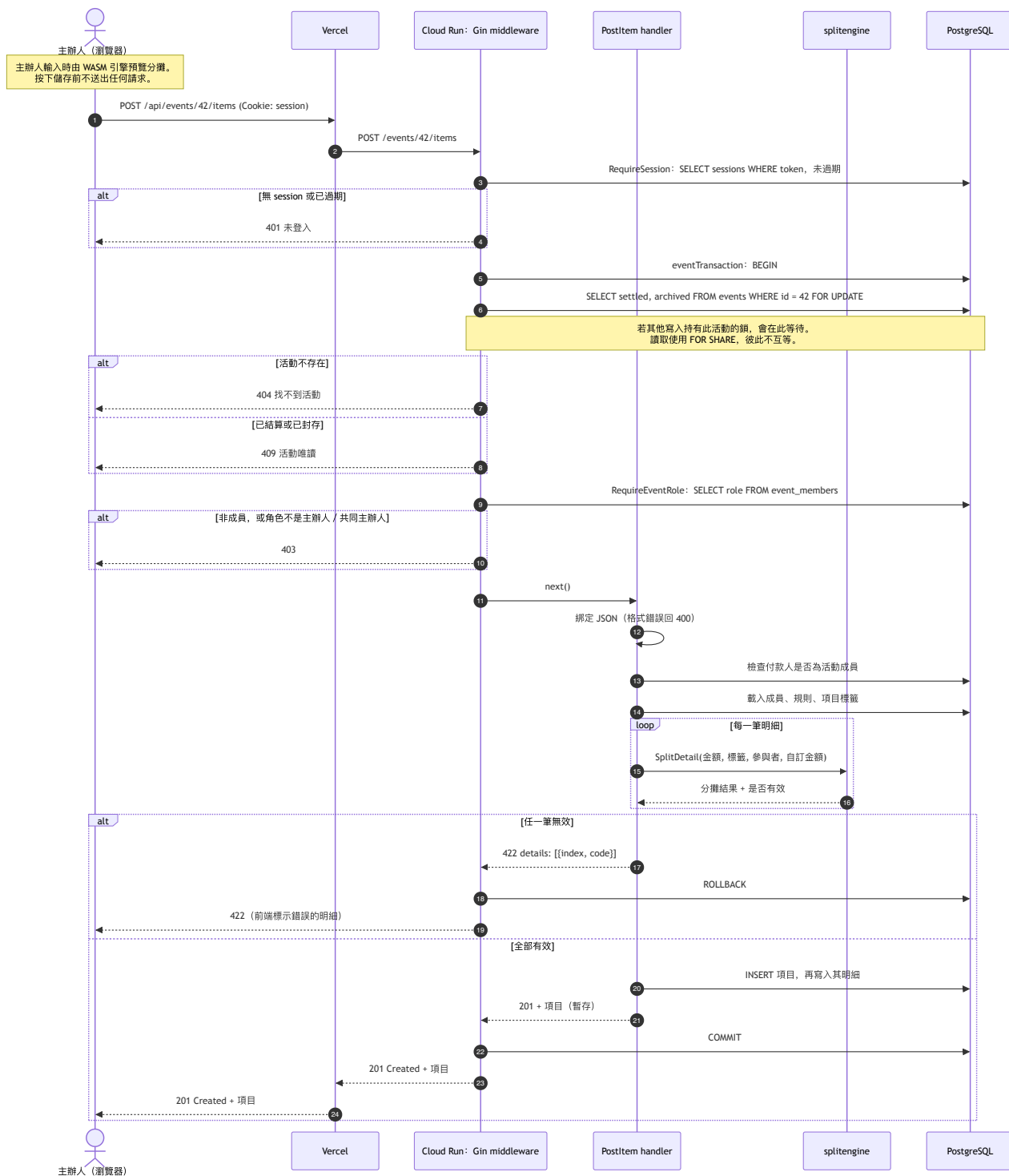
## 執行期視角：一個請求的完整路徑



1. **瀏覽器** → **Vercel**。前端呼叫同源的 `/api/...`，並附上 `session cookie`。
2. **Vercel** → **Cloud Run**。Next.js rewrite 將請求轉送到後端網址 (`API_URL`)。
3. **Google Front End** → **instance**。Google 終結 TLS 並路由到運行中的 instance。若沒有任何 instance (已縮減至零)，會先啟動一個，即冷啟動。
4. **Cloud Run** → **資料庫**。API 從 pgx 連線池借用連線，經掛載的 Unix socket 連到 Cloud SQL。只有 `POST /auth/google` 會另外呼叫 Google 驗證 ID token。
5. **回應**。`/events/{id}/...` 路由的回應會先暫存，等交易 commit 後才送出；若 handler 失敗則交易 rollback。因此用戶端不會對未儲存的資料收到成功回應。

## 循序圖：儲存一筆開銷 (`POST /events/{id}/items`)

儲存開銷會經過每一層：session、活動鎖、角色檢查、以分攤引擎驗證，以及資料庫寫入。



請求的主要成本在資料庫往返（兩筆明細的卡片約 14 次，每筆明細至少一次），而不是計算本身。因此當請求排隊等待連線池或活動鎖時，負載測試的延遲會上升（見第 3 節）。

## 建置與部署視角

- 後端 CI (PR 到 main)：race detector 單元測試、lint、API 端對端測試與 k6 負載測試，皆在一次性的 PostgreSQL 15 上執行。
- 後端 CD (push 到 main)：Terraform apply、經 Cloud SQL Auth Proxy 執行 migration、推送 image 到 Artifact Registry 並部署到 Cloud Run。
- 前端 (push 到 main)：Vercel 的 GitHub 整合自動建置並部署到正式環境。建置前 sync-engine 從 @go-split/engine 複製 engine.wasm，所以前端換用新版引擎只需升級 npm 套件。
- 引擎發布 (engine-vX.Y.Z tag)：建置、測試並發布 @go-split/engine。

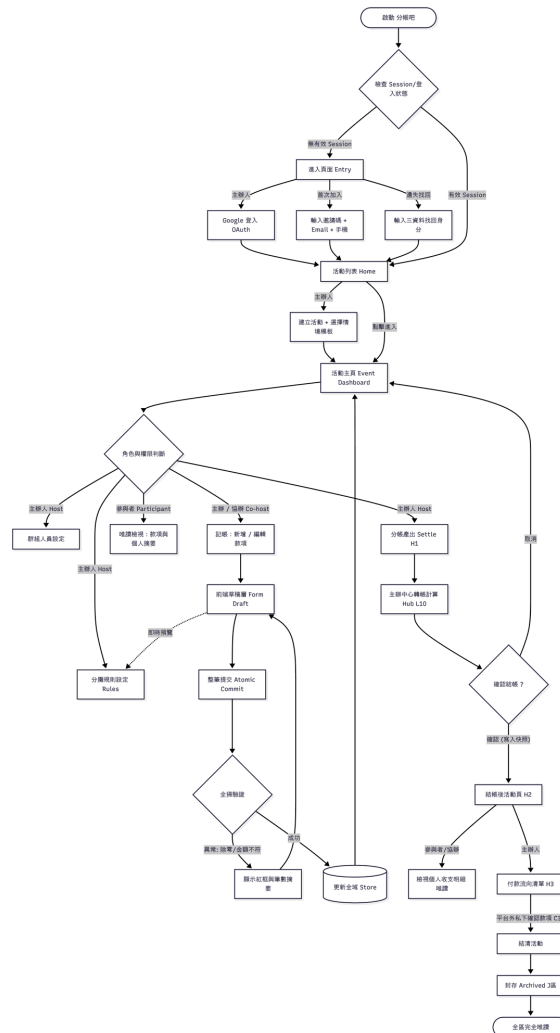
# 為什麼是這個架構

每個選擇都對應產品的一個限制：

- **預覽必須即時，且與結果一致** → 同一個 Go 引擎跑在兩處。主辦人調整規則或金額時要立刻看到每人分攤，等網路往返太慢；但若前端另寫一套 JS 計算，兩邊遲早算出不同數字。把伺服器的 Go 程式碼編譯成 WASM，預覽與正式結果就不會分歧，且以伺服器結果為準。
- **沒有自有網域** → 經 Next.js server 代理。前端在 go-split.vercel.app、後端在 Cloud Run 的 run.app 網址，兩者屬於不同網站。所有使用者都靠 session cookie 維持身分；若瀏覽器直接跨網站呼叫 Cloud Run，cookie 會成為第三方，Safari 等瀏覽器會擋掉。經由 Vercel 的 /api/\* rewrite 轉送，瀏覽器只看到同一個來源，cookie 維持第一方。
- **金額不能算錯，主辦人與共同主辦人可能同時編輯** → 交易與資料庫約束。同一活動的寫入在交易中鎖住活動列並序列化；「恰有一位主辦人」「已結算即唯讀」由資料庫約束保證；回應等 commit 後才送出，用戶端不會看到未儲存的成功。金額一律為新台幣整數元，不默默四捨五入。
- **結算後結果不能變** → 快照。結算時凍結輸入、結果、成員順序與引擎版本；之後即使引擎升級，舊活動仍讀取當時的快照。
- **用量集中在活動期間，平時幾乎閒置** → Cloud Run 縮減至零。一場旅行結束後可能數週沒有請求，按用量計費讓閒置成本趨近於零；代價是閒置後的第一個請求會冷啟動。Cloud SQL 選 db-f1-micro 也是同樣的取捨，連線上限見第 3 節。
- **小團隊、時程有限** → 全用託管服務，基礎設施即程式碼。Vercel、Cloud Run、Cloud SQL 免去維護；Terraform 與 GitHub Actions 讓每次合併到 main 都自動部署，不依賴個人手動操作。

## 2. 介面與使用者體驗設計

系統有兩種角色：**主辦人**以 Google 登入並負責記帳；**成員**可以以訪客身分加入、不需帳號，主要查看自己的分攤。下圖是完整的使用者流程；本節從中挑出決定分帳是否順利的三個流程：加入、設定規則、結算。



## 2.1 不需帳號即可加入

分

帳吧

輕鬆分帳，活動費用一目了然

使用 Google 帳戶登入

或

Email

you@example.com

手機

09xx-xxx-xxx

活動邀請碼

8MSK-7G

下一步

< 填寫加入訊息

參與者姓名

你的名字

人員條件

#吃素

#大人

#小孩

#晚到

#海鮮過敏

#不喝酒/開車

#自行前往

#需搭主辦的車

其他備註

例：會晚 20 分鐘到

加入

## 設計決策

- 訪客不需帳號。QR code 與連結會開啟 `/login?invite=CODE`，邀請碼已預先填好。訪客只需提供 email 與手機號碼。提供 Google 登入但非必要。
- 成員自行表明條件。第 2 步顯示該活動自己的條件清單（#吃素、#小孩 ...）。清單透過以邀請碼查詢的公開 API 載入，只回傳活動名稱與清單。主辦人不需要知道誰吃素。

## 2.2 由規則決定分攤

三

商品 攤位 分攤規則 主頁 水單

分攤規則

烤肉工具 露營工具 免洗餐具 清潔用品 場地費 設備租借費 交通費

人員條件 (人員定義用)

修改會重算已記錄的金額

「肉品」已有 1 筆支出。分攤結果在結清前都會即時重算，改了規則，這些支出的金額和成員看到的分攤都會跟著改變。

取消 仍要修改

肉品 已被使用

分攤規則

吃素 1 人 不計入

小孩 1 人 權重 ×0.5

晚到 0 人 權重 ×0.5

其他人員 權重 ×1

< 新增款項

本筆款項合計 NT\$ 500

明細

雞腿 10 支 500

肉品

其他備註

分攤預覽

Will 其他人員 NT\$ 200

Ben 其他人員 NT\$ 200

Coco 小孩，權重 ×0.5 NT\$ 100

Amy 不計入 (#吃素) NT\$ 0

為什麼關鍵。團體分帳的難處不在算術，而在「誰分攤什麼」：吃素的人不付肉錢，小孩付一半。逐筆手動分攤最容易出錯，所以產品把這件事交給主辦人設定一次的規則。

## 設計決策

- **範本優先。** 選擇「烤肉」或「露營」會預先建立項目標籤、成員條件與規則（例如肉品：#吃素 不計入、#小孩  $\times 0.5$ ）。多數主辦人從不需要打開規則編輯器。
- **為明細加上標籤，儲存前即可看到分攤。** 主辦人輸入名稱、金額與標籤，表單會隨輸入即時預覽每位成員的分攤與原因。以 NT\$ 500 的肉品為例：Will 與 Ben 各付 200，Coco 付 100（小孩，權重  $\times 0.5$ ），Amy 顯示「不計入（#吃素）」。
- **瀏覽器執行伺服器的引擎。** 預覽與規則編輯器的檢查都使用編譯成 WASM 的 Go 引擎，因此主辦人儲存前看到的，就是 API 存下的結果。
- **變更既有結果前先警告。** 結算前分攤會即時重新計算，所以編輯已被開銷使用的規則時，會跳出對話框說明有幾筆明細會改變。已使用的標籤與規則沒有刪除按鈕，API 也會以 409 拒絕刪除。

## 2.3 結算與每位成員看到的內容

| 代墊金額      | 分帳產出     |
|-----------|----------|
| NT\$ 0    |          |
| Coco      | 參與者      |
| #小孩       |          |
| 分攤金額      |          |
| NT\$ 242  |          |
| 代墊金額      |          |
| NT\$ 0    |          |
| 人員付款流向    |          |
| Will      | 主辦者      |
| → 付給 Amy  | NT\$ 332 |
| ← 收自 Ben  | NT\$ 840 |
| ← 收自 Coco | NT\$ 242 |
| 應收回       | NT\$ 750 |
| Amy       | 參與者      |
| ← 收自 Will | NT\$ 332 |
| 確認結帳產出    |          |

| <        | 款項細項      |
|----------|-----------|
| 豬五花 2 盒  | NT\$ 680  |
| 肉品       |           |
| 分攤人員     |           |
| Amy      | 不計入 (#吃素) |
| 玉米、青椒、香菇 | NT\$ 320  |
| 蔬菜       |           |
| 分攤人員     |           |
| Amy      | NT\$ 120  |
| 白飯 + 吐司  | NT\$ 150  |
| 主食       |           |
| 分攤人員     |           |
| Amy      | NT\$ 43   |

為什麼關鍵。結算是唯一無法復原的步驟，而且會產生大家實際要付的金額。

## 設計決策

- **預覽即將凍結的確切結果。** 頁面顯示每筆開銷、每人應付與已付金額及轉帳，確認時再問一次。伺服器結算前會再檢查每一筆明細，有任何無效就回 422，不凍結任何資料。
- **所有轉帳都經過主辦人。** 每位成員只有一筆轉帳：付給主辦人，或由主辦人付給你，比成員之間的最少轉帳圖更容易理解。
- **成員只看到自己的數字。** 成員看得到每筆開銷金額，但只看到自己的分攤；不計入時會說明原因（「不計入（#吃素）」）。主辦人看得到所有人的分攤。

## 3. 擴展性設計

### 預期規模

Go-Split 服務的是小團體：一趟旅行、一場烤肉、一頓聚餐。一個活動有數人到數十位成員，以及數十到一兩百筆開銷明細。流量是突發性的：主辦人儲存後成員會查看自己的分攤，主辦人結算時所有人會同時查看。單一活動很少每秒超過幾次儲存。

### 設計上為擴展所做的事

- **Cloud Run 上的無狀態 API**。Session 存在 PostgreSQL，任何 instance 都能處理任何請求。Cloud Run 在 0 到 5 個 instance 之間擴展（每個 1 vCPU、512 MiB）。
- **預覽在瀏覽器執行**。主辦人輸入時由 WASM 引擎計算分攤，只有最後的儲存會送到伺服器。
- **靜態資源由 Vercel CDN 提供**。前端頁面與 engine.wasm 從 CDN 載入；/api/\* 則原封不動轉送到 Cloud Run，不做快取，所以 API 的擴展仍由 Cloud Run 與資料庫決定。實測經 Vercel 轉送每個請求多約 41 ms (P50) / 53 ms (P95)。
- **每個活動的寫入序列化**。同一活動的儲存、加入與結算在交易中逐一執行。這確保結果正確，也限制了單一活動每秒能承受的寫入數（見下文）。
- **結算快照**。結算後的讀取來自凍結的快照，不再重新計算。
- **PGO 流程**。正式環境的 CPU profile 上傳到 GCS，可用於 go build -pgo。

### 負載測試

以 k6 進行負載測試。CI 每次以 make load-smoke (GET /events, 20 RPS 持續 30 秒) 做回歸檢查。容量則以 make load-prod 對正式環境量測：使用真實的 session 與活動，對五個 endpoint 送出日常讀取流量。它略過儲存與結算，因為建立這些資料需要直接存取資料庫，而結算會永久凍結活動。

#### 正式環境結果 (2026-10-01)

以 make load-prod 直接對 asia-east1 的 Cloud Run 執行（不經 Vercel）。測試活動有 6 位成員、16 筆開銷，只有讀取；每一階段爬升 60 秒、維持 90 秒。

| RPS | P95           | 最慢的 endpoint (P95)    | 錯誤 / 5xx | 丟棄的 iteration | 請求數    |
|-----|---------------|-----------------------|----------|---------------|--------|
| 50  | 64.5 ms       | /me/details (71.4 ms) | 0 / 0    | 0             | 6,375  |
| 75  | 52.9 ms       | /me/details (58.8 ms) | 0 / 0    | 0             | 9,562  |
| 100 | <b>7.06 s</b> | /events/{id} (7.23 s) | 0 / 0    | 1,311         | 11,438 |

50 與 75 RPS 時都遠低於 500 ms 的目標。到 100 RPS 時沒有任何錯誤，但請求開始排隊，P95 急遽升到約 7 秒，因此沒有繼續測 150 與 300。所有 endpoint 一起變慢，代表瓶頸是共用資源：5 個 1 vCPU 的 instance，或約 20 條資料庫連線；測試時無法取得 GCP 指標確認是哪一個。此外也進行過多輪測試（包括限制每個 instance 的連線數），結果沒有明顯變化。

正式環境的讀取容量約 75 RPS，低於 300 RPS 的參考值，但仍遠高於預期規模（單一活動很少每秒超過幾次儲存）。

# 已知限制與瓶頸

## 資料庫連線（主要限制）

- Cloud SQL db-f1-micro 約允許 25 條連線，其中幾條保留給 Cloud SQL 自己。
- Cloud Run 上限為 5 個 instance (cloud\_run\_max\_instances)，每個 instance 以 DB\_MAX\_CONNS 限制為 4 條連線，共 20 條，剛好低於資料庫上限。CD 的 migration 步驟經 Cloud SQL Auth Proxy 還需要一條，餘裕很小；若提高 instance 上限或每個 instance 的連線數，資料庫會拒絕連線。

可行做法，由便宜到昂貴：

1. 換到更大的 Cloud SQL 方案，允許更多連線。
2. 在資料庫前加上連線池代理 (PgBouncer，或 Cloud SQL 託管的 pooler)。

## 其他限制

- **單一活動的寫入逐一執行。** 熱門活動在每秒 20 次儲存下維持 500 ms 以內，前提是連線池限制了並行數。這遠高於實際使用量，但一個很大、很活躍的活動仍會排隊。
- **分攤在讀取時計算。** 結算前，/shares 與 /me/details 每次請求都會重算所有明細，成本隨活動的明細數成長。實際規模下沒有問題；只有非常大的活動才值得快取。
- **冷啟動。** 縮減至零代表閒置後的第一個請求會啟動新 instance，在提供服務前先執行 migration 與範本 seeding。嘗試量測時，閒置 21 分鐘後仍有 instance 在執行（第一個請求 90 ms），所以尚未取得冷啟動數字。
- **單一 zone 的資料庫。** availability\_type = "ZONAL"。zone 故障時資料庫會停擺直到恢復；已開啟備份與時間點還原。

## 4. 容錯與韌性設計

Go-Split 處理的是金錢，所以設計上優先確保**故障時的正確性**：失敗的請求不能留下寫到一半的資料，已結算的活動永遠不能改變。

### 總覽

| 故障                     | 結果                            | 位置                                                |
|------------------------|-------------------------------|---------------------------------------------------|
| 請求中途 handler 出錯或 panic | 交易 rollback；用戶端收到錯誤，絕不會收到假的成功 | internal/events/transaction.go、server.go recovery |
| 同一活動同時有兩筆寫入            | 由活動列的鎖序列化；第二筆等待               | eventTransaction (FOR UPDATE)                     |
| 寫入已結算的活動               | API 拒絕 (409)，資料庫 trigger 再擋一次 | transaction.go、migration 0009                     |
| Google 登入緩慢或故障         | 逾時 10 秒後回 504；其他上游錯誤回 502     | internal/auth/google.go                           |
| Profile 上傳到 GCS 失敗     | 記錄 log 後略過；請求不受影響             | internal/profiling                                |
| 啟動時資料庫無法連線             | 程序結束；Cloud Run 重啟 instance    | server.go (log.Fatalf)                            |
| Instance 關閉（部署、縮減）     | 進行中的請求最多有 10 秒完成              | server.go graceful shutdown                       |
| 資料庫 instance 或 zone 失效 | 停機直到恢復；可從備份 / PITR 還原資料       | Terraform deploy/gcp/main.tf                      |

### 錯誤處理

**後端**。每個 handler 都明確選擇狀態碼，並以相同的 JSON 格式 `{"error": "..."}`  回應。

Panic 由 Gin 的 recovery middleware 捕捉、連同 stack 記錄 log，並回應 500。活動交易的 deferred rollback 仍會執行，所以 panic 不會留下部分寫入。

**前端**。readApiError 將錯誤格式轉成訊息，並附上 422 / 409 的代碼：error (code1、code2)。頁面以錯誤橫幅（結算、規則）、toast（成員）或表單下方的 inline 訊息（建立活動、項目明細）顯示錯誤。規則頁採樂觀更新，若儲存失敗則重新向伺服器取得資料，讓畫面與已儲存的狀態一致。

### 資料一致性

由三層把關。

**1. 每個活動請求一個交易**。一個請求的所有寫入都在同一個交易中：全部成功才 commit，任何錯誤都整筆 rollback。回應等 commit 後才送出，所以用戶端不會對未儲存的資料收到成功。

2. 資料庫也負責把關規則。API 檢查不是唯一的防線：資料庫本身也以 unique index、check 約束與 trigger 限制欄位，例如每個活動恰有一位主辦人、已結算的活動不能再寫入。即使 handler 有 bug，也改不了已結算的活動。

3. 金額精確，結算凍結。金額一律以整數元儲存；結算後所有結果都凍結，不再改變。

## 資料庫韌性

Cloud SQL 有每日備份與時間點還原、僅允許加密連線，並自動擴充磁碟。它在單一 zone 執行、沒有備援，所以 zone 或 instance 故障就會停機直到恢復（見第 6 節）。

## 可觀測性與其限制

我們依賴 Google Cloud 預設提供的功能，沒有額外工具：

| 訊號       | 能取得的內容                                                          | 來源                         |
|----------|-----------------------------------------------------------------|----------------------------|
| 請求 log   | 每個請求一筆：method、URL、狀態碼、延遲、大小、user agent、用戶端 IP、trace id、instance | Cloud Run，自動               |
| 應用程式 log | 伺服器寫到 stdout / stderr 的內容（logrus，純文字）                           | Cloud Run，自動               |
| 指標       | 請求數與延遲、instance 數、CPU、記憶體、啟動延遲、並行數、流入 / 流出位元組                   | Cloud Run，內建               |
| Trace    | 每個取樣請求一個 span，每個 instance 每 10 秒最多 1 個請求，免費                     | Cloud Run → Cloud Trace，自動 |
| Profile  | 每 10 分鐘一次 CPU profile，用於 PGO                                    | 自己寫的上傳程式，存到 GCS            |

這能看出請求失敗了、何時、在哪個 instance，但多半看不出為什麼，或哪個 endpoint 變慢：

- **請求 log 沒有內容。** 不含請求或回應內容，也沒有呼叫者身分。看得到 POST /events/42/items 失敗，但看不到送了什麼、是誰送的。
- **應用程式 log 還補不上這個缺口。** 多數 500 不會記錄原因。log 是純文字，所以 Cloud Logging 儲存時沒有 severity；沒有 logging.googleapis.com/trace 欄位，也就不會歸到對應的請求底下（第 6 節）。
- **Log 只保留 30 天。** 這是 \_Default log bucket 的預設值；可提高到最多 3,650 天，但要多付儲存費用。一個月後才回報的問題就沒有 log 可查。
- **指標無法依 endpoint 拆分。** 請求數與延遲可以依狀態碼與 revision 分組，但沒有 URL 路徑的 label，所以「P95 上升」看不出是哪個路由。Log-based metric 可以加上路由，但我們的路徑含有 id (/events/42/items)，需要先正規化。
- **看不到程序內部。** 沒有量測等待連線池或活動鎖的時間、查詢耗時，或 Go runtime 健康狀況（goroutine、GC）。第 3 節發現這些等待正是延遲的主因。Cloud SQL 自己的指標能部分涵蓋資料庫端。
- **Trace 只顯示總時間。** Cloud Run 自動產生的 trace 每個請求只有一個 span，看不出是哪個查詢或鎖花了時間。子 span 需要 OpenTelemetry instrumentation，Cloud Trace 會另外計費。

## 5. 資安設計

Go-Split 儲存誰欠誰多少錢，以及成員的姓名、email 與手機號碼。對這個產品最重要的四個風險，以及它們對應的 OWASP Top 10 類別：

1. 有人讀取或修改不屬於自己的活動 (A01)；
2. 成員做了超出角色權限的事，例如編輯別人的開銷或進行結算 (A01)；
3. 已結算的結果事後被改變 (A04、A08)；
4. 個人資料外洩，尤其是訪客的 email 與手機號碼 (A01、A02、A07)。

以下先列出各 OWASP 類別的狀態，再說明已做的處理，最後列出缺口與改進方式。

### OWASP Top 10 (2021)

| 類別             | 對應風險  | 狀態                               |
|----------------|-------|----------------------------------|
| A01 存取控制失效     | 1、2、4 | 部分處理 (缺口：訪客冒用)                   |
| A02 加密機制失效     | 4     | 已處理                              |
| A03 Injection  | —     | 已處理                              |
| A04 不安全設計      | 3     | 已處理                              |
| A05 安全設定錯誤     | —     | 部分處理 (缺口：沒有 CSP)                 |
| A06 易受攻擊的元件    | —     | 缺口：沒有相依套件掃描                      |
| A07 識別與驗證失效    | 4     | 部分處理 (缺口：沒有 rate limiting)       |
| A08 軟體與資料完整性失效 | 3     | 部分處理 (缺口：Actions 未固定 commit SHA) |
| A09 記錄與監控失效    | —     | 部分處理 (缺口：500 沒有記錄原因)             |
| A10 SSRF       | —     | 不適用：唯一的對外呼叫指向固定的 Google 網址       |

粗體為與上述四個風險直接相關的類別。

### 已做的處理

- **A01 存取控制**。每個 `/events/{id}` 路由都經過 `RequireSession` (無 session 回 401) 與 `RequireEventRole` (非成員或角色不符回 403)。主辦人可管理整個活動；共同主辦人只能編輯自己建立的開銷；成員只能讀取。項目與成員的查詢都同時以物件 id 與 `event_id` 篩選，其他活動的 id 查不到資料。已結算的活動在 API (409) 與資料庫 trigger 兩層都拒絕寫入。
- **A02 加密**。瀏覽器到 Vercel、Vercel 到 Cloud Run 全程 HTTPS；Cloud SQL 只接受加密連線。Session token 與邀請碼都以 `crypto/rand` 產生。Session cookie 為 `HttpOnly`、`Secure`、`SameSite=Lax`，前端從不接觸 token。資料庫密碼與 Google client ID 存在 Secret Manager；CD 以 Workload Identity Federation 驗證，沒有長期金鑰。
- **A03 Injection**。所有 SQL 都使用 `pgx` 的 `$n` placeholder，只有伺服器端常數會拼進 SQL。前端所有使用者內容都經過 JSX escape，沒有 `dangerouslySetInnerHTML` 或 `eval`。
- **A04 安全設計**。金額一律由伺服器以自己的引擎重算，忽略用戶端算出的總額；結算後結果來自記錄引擎版本的凍結快照。邀請碼在活動結算後失效 (410)。
- **A05 設定**。容器以 FROM scratch 建置，沒有 shell。Cloud Run 的 service account 只有 Cloud SQL client、兩個 secret 與 profiling bucket 的權限。Cloud SQL 沒有允許任何網路，連線都經過有

IAM 檢查的 connector。Vercel 上沒有任何 secret。

- **A07 驗證。** Google 登入時自行檢查 ID token 的 aud、iss、exp 與 email\_verified。登入時伺服器一律建立新的 session token，不接受用戶端提供的，因此不會有 session fixation。Session 30 天到期，每個請求都在 SQL 中檢查；登出會刪除 session。
- **A08 完整性。** CD 以 Workload Identity Federation 部署；結算快照記錄引擎版本，引擎升級不會改變已結算的結果。
- **A09 記錄。** Cloud Run 記錄每個請求並提供請求指標；記錄 log 前會移除 token。

## 缺口與改進

| 缺口                           | 風險                                                                           | 改進方式                                                             |
|------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------|
| A01 訪客可被冒用                   | 訪客身分只以未驗證的（email、手機）識別。任何人只要有任一有效邀請碼，加上受害者的 email 與手機，就能取得該訪客在所有活動中的 session | 將訪客限定在單一活動，或在沿用既有訪客前寄送一次性驗證碼到 email                              |
| A05 沒有 CSP 與 frame-ancestors | 只設定了 Cross-Origin-Opener-Policy，app 可被其他網站嵌入 frame，XSS 也少了一層防護               | 在 next.config.ts 加上 Content Security Policy 與 frame-ancestors    |
| A06 沒有相依套件掃描                 | CI 只跑 golangci-lint，Go 與 npm 套件的已知漏洞不會被發現                                    | 啟用 Dependabot，在 CI 執行 govulncheck、npm audit 與 gosec              |
| A07 沒有 rate limiting         | /auth/join、/auth/recover 與公開的邀請碼查詢都沒有限流；邀請碼約 30 bits 的熵，大規模列舉可行              | 對 /auth/* 做 per-IP 限流（經 Vercel 轉送時以 X-Forwarded-For 辨識來源），並加長邀請碼 |
| A08 Actions 以 tag 固定         | 第三方 Action 的 tag 被改寫時，CD 會執行被竄改的程式碼                                          | 以 commit SHA 固定 Actions                                          |
| A09 500 沒有記錄原因               | 正式環境的錯誤只看得到失敗，看不到原因，也無法連到對應請求                                                | JSON log，加上 severity、trace id 與錯誤原因（見第 6 節）                      |

## 6. 技術債紀錄

以下是開發期間刻意接受、或尚未處理的主要取舍。資安相關的未解決風險見第 5 節。

| 技術債                   | 做了什麼                                                                                      | 為什麼                                                      | 修正方向                                           |
|-----------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------|------------------------------------------------|
| 資料庫連線接近上限             | 每個 instance 4 條連線 (DB_MAX_CONNS)，5 個 instance 共 20 條，接近 db-f1-micro 約 25 條的上限             | 這個階段以成本優先，使用最小的方案                                        | 換更大的 Cloud SQL 方案或加連線池代理                       |
| 資料庫沒有高可用              | 單一 zone、沒有 standby；有每日備份與時間點還原                                                            | 這個階段以成本優先，standby 約讓資料庫費用加倍                              | 需要可用性時改為 REGIONAL                              |
| 每個 API 請求都經 Vercel 轉送 | 瀏覽器呼叫 Vercel 上的 /api/*，再由 Next.js server 轉送到 Cloud Run；實測每個請求多約 41 ms (P50) / 53 ms (P95) | 小專案不想購買自有網域；前後端不同網站時 session cookie 會成為第三方而被瀏覽器擋掉（第 1 節） | 購買網域，將前端與 API 放在同一網站的子網域下，瀏覽器即可直接呼叫 Cloud Run  |
| 錯誤沒有記錄原因              | 多數 500 只回傳固定訊息；log 為純文字，沒有 severity，也沒有連到請求的 trace id                                     | 開發期間本機除錯就夠用                                              | JSON log，加上 severity、trace id 與錯誤原因            |
| 前端錯誤處理與監控             | 許多載入錯誤被空的 catch 吞掉；沒有處理 401；瀏覽器端沒有錯誤回報                                                    | 從原型延續，優先完成功能                                             | 共用錯誤與 toast 路徑、401 導向登入；規劃導入 Sentry（關閉 PII 收集） |
| 前端沒有測試                | 沒有 E2E 測試，src/components/ui 的 12 個共用元件 (design system) 也沒有單元測試                            | 從原型延續，優先完成功能                                             | 為共用元件寫單元測試，並為建立活動與加入流程寫 E2E smoke test         |
| 前端沒有錯誤頁面              | App Router 沒有 error.tsx、global-error.tsx 或 not-found.tsx；頁面錯誤或不存在的路由只會顯示 Next.js 預設畫面     | 從原型延續，優先完成功能                                             | 加上錯誤與 404 頁面，提供重試與回到首頁                         |
| 部署沒有把關                | 後端 CD 不等 CI 就部署，Terraform 以 -auto-approve 套用；前端沒有任何 CI，push 到 main 即上線                    | 預期所有變更都經過審查的 pull request                                | 部署依賴 CI、terraform plan 需核准；前端加上 lint、型別檢查與建置   |
| Migration 只能往前        | 自製 runner 依檔名套用、沒有 down migration；每次 instance 啟動都會再執行                                     | 最簡單的做法，migration 工具列為後續工作                                | 改用 goose 或 golang-migrate，先擴充後收縮，只在 CD 執行      |